

Localization Review Console — Multi-Language Expansion (v2)

Product Requirements Document Status: Draft · Owner: Aashma · Builds on: Localization Review Console v1 (shipped, Nepali-only)

1. Summary

The Localization Review Console is an internal, browser-based tool that lets a language reviewer listen to a piece of localized voice-over, read it against its English source, and attach precise notes — highlighting problem spans and flagging pronunciation or phrasing issues — then export those notes as a clean spreadsheet for the production team.

Version 1 shipped as a single-language (Nepali) tool. This document specifies **v2**, which generalizes the tool to multiple languages (adding Bengali, Tamil, and Malayalam alongside the already-supported Devanagari languages), ingests the studio's real multi-sheet game-strings workbooks directly, and lands four workflow improvements requested after v1 usage. v2 is an **extension of the existing codebase**, not a rewrite: the review, highlight, note, and export mechanics already work and are language-agnostic; the work is generalizing the language-specific seams and improving the reviewer's loop.

2. Background: what v1 already does

v1 is a fully client-side React/TypeScript/Vite app. A reviewer loads a review spreadsheet and a folder of audio clips; the tool shows each string as **English → target language → phonetic romanization** with a matched audio player, and lets the reviewer select text to attach a colored note (orange for phrasing, red for serious) or flag an entire row. Notes export to a **new** spreadsheet. The tool never modifies its inputs and makes no network calls.

The important property v1 established — and v2 preserves as a hard principle — is that the tool is **read-only on all source content and emits only reviewer notes**. Nothing is uploaded; the source strings and audio never leave through the tool.

3. Problem statement

The studio localizes training content into nine languages and must have native reviewers catch pronunciation and phrasing errors in the generated voice-over before it ships. Today that review is slow and risky:

- **It only supports one language.** v1 is hardcoded to Nepali. Every other language is unserved, yet the same review need exists across all of them.

- **It requires manual, error-prone setup.** The reviewer must hand-pick and load the correct spreadsheet and the correct audio folder. The real source workbooks hold *all* languages as columns across *many* submodule sheets — the tool can't read that shape, so someone has to pre-slice files by hand.
- **Content confidentiality is a real constraint.** Reviewers are external-facing contractors; unreleased game content cannot leak. There is no controlled way to give a reviewer *only* what they need to see.
- **The review loop has friction.** "Progress" is inferred from mere navigation rather than a deliberate check; audio restarts from zero the moment a reviewer starts typing a note; and highlighted spans lose their visual marking on export.

4. Goals and non-goals

Goals

- Support reviewing any of the studio's languages, delivering **Bengali, Tamil, and Malayalam** in this release (Hindi comes essentially free — see §7).
- Let the tool **ingest the real multi-sheet game-strings workbook** and extract only what a given reviewer needs (Key, English, and their one target language).
- Make the reviewer's loop **deliberate and low-friction**: manual completion checks, audio that survives note-taking, and highlights that persist visibly into the export.
- Preserve the **read-only / no-leak** guarantee as a stated design principle, and reinforce it via per-language extraction.

Non-goals (this release)

- No editing or change-request workflow. A separate team tool already lets reviewers propose changes and regenerate audio; this tool is **notes only**.
- No backend, database, accounts, or cross-device saved progress. Deferred to a defined future phase (§9).
- No Arabic/Urdu (right-to-left) support yet. Their script direction and transliteration needs are a separate, larger effort (§9).

5. Users

The reviewer (primary). A native speaker of one target language, working in a controlled in-office environment. They care about hearing each clip, comparing it to the source, and recording precise notes quickly. They should see only their language and never hold raw source files.

The internal coordinator (secondary). A studio team member who sets up a review session — loads the game-strings workbook and the audio for a submodule into the tool — and later collects the exported notes.

6. Design principle: isolation by construction

Confidentiality is enforced primarily by the *environment* (a locked-down in-office remote desktop with no external network or removable storage). The tool's job is to **add no leak vectors and minimize what a reviewer can see or extract**:

- The tool is client-side with **zero network calls** — source content is never transmitted anywhere.
- The tool offers **no way to re-download the source workbook or audio**; the only export is reviewer notes.
- **Per-language extraction** means a reviewer's session contains only Key, English, and their single target-language column — not the other languages or internal columns.

This principle should be stated explicitly in the tool and its documentation, because "we only ever read inputs and emit notes" is both the security story and the reason the tool is safe to hand to contractors.

7. Requirements

F1 — Multi-language via language profiles

Replace v1's hardcoded "Nepali" concept with a **language profile keyed by ISO code** (`ne` , `hi` , `bn` , `ta` , `ml` , ...). Each profile declares its display name, script, text direction, display font, and optional romanization engine. This single change turns every language-specific seam — column selection, audio folder, panel label, font — into a lookup.

The studio's data makes this clean: spreadsheet columns already embed the code (`Nepali(ne)` , `Bangla(bn)` , `Tamil(ta)`) and audio folders are named by the same code (`NE` , `BN` , `TA`). One code joins sheet, audio, and profile.

In scope for this release: Nepali (`ne`) and Hindi (`hi`) — both Devanagari, already handled by the existing romanizer — plus **Bengali (`bn`)**, **Tamil (`ta`)**, **Malayalam (`ml`)**, each a distinct script requiring its own font and (optionally) romanizer.

F2 — Language selector

A dropdown lets the reviewer choose the active language. When a submodule's audio is loaded, the dropdown reflects the languages actually present. Selecting a language sets the target column read from the sheet, the audio folder played, and the display font — everything follows from the profile.

F3 — Multi-sheet workbook ingestion and extraction

The tool must load the studio's real workbook (one module file with many submodule sheets and all-language columns) and:

- Treat any sheet containing both a `Key` column and an English (`en`) column as a reviewable submodule; skip the rest (Summary, issue-tracking tabs).

- Match the target-language column by its **ISO code in parentheses**, not its display name (so `Bangla(bn)` resolves correctly).
- Extract **only Key, English, and the selected language** into the in-memory review set — the extraction that both simplifies the reviewer's view and enforces confidentiality.

All of this happens in memory against a read-only copy; the workbook file is never written.

F4 — Language-aware audio auto-detection

Dropping a submodule's audio folder yields subfolders per language (`AR BN EN HI ML NE TA TAG UR`), each containing clips whose filename stems equal the string Keys. The tool must **index audio scoped by language code**, because the same stem exists in every language folder — a flat index would collide and play the wrong language. Matching a string to a clip is then keyed by (*active language code, Key*).

Known case to decide: some strings have regional/context variant clips (e.g. `_KSA` suffix, parallel `_Retail` folders). This release should at minimum not mis-match them; surfacing variants for review is a candidate follow-up.

F5 — Manual completion checks

Today a string is marked "seen" simply by navigating to it. v2 **decouples navigation from completion**: a string counts as reviewed only when the reviewer explicitly ticks a checkbox. Progress indicators reflect manual checks, giving an accurate, deliberate measure of review coverage.

F6 — Audio continuity during note-taking

Starting a note must no longer restart audio from the beginning. Provide a **user setting** with two modes, because reviewers differ: *keep playing* (transcribe while listening) or *pause on note-open and resume from the same position on close*. Either way, audio never jumps back to zero.

F7 — Highlights preserved in the export

Highlighted spans must carry their color (orange/red) into the exported spreadsheet, marking the **exact words** the reviewer flagged — not just the whole cell. This is achievable because highlights are already stored as precise character offsets; the export renders each target cell as rich text with the flagged runs colored by severity. (A colored *background* behind mid-sentence words is not expressible in the spreadsheet format; colored text on the exact span is, and is the chosen approach. A future Word/HTML export could add true background highlighting if needed.)

F8 — Romanization as an optional aid

Romanization is a crutch for reviewers who read Latin more fluently than the script, and is unnecessary for others. Make the romanization panel a **toggle**. Prefer a **transliteration column supplied by the source sheet** when present (the workbook already carries one

for some languages); fall back to a generated romanization where an engine exists. Critically, this makes romanization a *progressive enhancement* — the new languages can ship without blocking on building three new transliteration engines.

8. Success metrics

- **Coverage:** number of languages reviewable rises from 1 to 5 (ne, hi, bn, ta, ml).
- **Setup effort:** reviewer manual file-picking steps drop to near zero — load once, select language, review.
- **Review accuracy:** completion count reflects deliberate checks (F5), so "100% reviewed" means every string was actually verified.
- **Loop friction:** zero audio restarts during note-taking (F6); 100% of highlighted spans visible in the export (F7).
- **Confidentiality:** a reviewer's session and export contain only their one language — verifiable by inspection.

9. Scope and phasing

Phase 1 (this release): F1–F8 above, on the existing client-side architecture, in-memory state. Ships Tier 2 languages plus the four workflow fixes.

Phase 2 (deferred, trigger-based): Saved progress and a backend. The stepping stone is browser-local persistence (progress survives a refresh with zero infrastructure). A true backend — reviewer accounts, cross-device resume, assignment, centralized note collection, audit trail — is warranted **once** the team needs assignment across many reviewers or central reporting, and pairs naturally with the planned in-office hosting. Naming that trigger rather than building speculatively is the deliberate call.

Phase 3 (future): Right-to-left languages (Arabic, Urdu), which require text-direction handling, bidi-safe span selection, matching fonts, and a different transliteration approach.

10. Risks and tradeoffs

- **Romanizer build-vs-buy.** New scripts need transliteration. Making romanization optional (F8) removes it from the critical path; where we do want generated output, we choose per-script between hand-rolled tables (matching the existing Devanagari engine's style) and an existing library. Decision deferred to implementation, de-risked by the toggle.
- **Exact-span highlighting is format-limited.** Spreadsheets can color exact *text* but not the *background* behind a mid-cell span. F7 accepts colored text; a richer format is a future option if reviewers ask for a true highlighter look.
- **Audio variants.** Regional/retail alternate takes mean Key→clip is not strictly 1:1. Phase 1 must avoid mis-matching; full variant review is a candidate follow-up.

- **Deferring the backend.** In-memory state means a browser refresh loses progress. Accepted for Phase 1; local persistence is the cheap mitigation before any backend investment.

11. Open decisions

1. Reviewer works one submodule at a time (confirmed) — should the exported notes be **grouped per submodule sheet** (mirroring the source, easier for devs to trace) or a single flat sheet?
2. Audio **variant policy**: ignore `_KSA` / `_Retail` variants in Phase 1, or surface them for review?
3. Romanization **default state** per language: on or off when a reviewer first opens a language?